Tallinn University

School of Digital Technologies

DTLGM.DT Digital Learning Games

Designing an XR Drone Training Simulator

Master's Thesis

Author: Jian Gong

Supervisor: Martin Sillaots

Tallinn 2026

Abstract

Automation is progressively displacing skilled technical labor, however, current robotic systems remain limited in replicating complex tasks, leaving a persistent need for human operators. At the same time, such skills are difficult to acquire, requiring prolonged training and the development of muscle memory. This creates a gap between the continued demand for skilled operators and the accessibility of training pathways.

This paper presents a design-based research study investigating Extended Reality (XR) as a medium for lowering the learning barrier and increasing engagement in complex skill acquisition, using First Person View (FPV) quadcopter piloting as a representative case domain.

This study is structured around three primary objectives: (1) the simulation of FPV quadcopter flight physics. (2) the design of a human-AI co-training framework (3) the intraction instruments for a high-fidelity XR prototyping. In particular PID and RL (Reinforcement Learning) are used in the co-training AI, and Geographic Information System (GIS) data integrated to support the environmental representation.

The contribution of this work is a structured design framework that links physical simulation, AI-assisted co-training, and XR-based interaction design, aimed at reducing the learning curve and improving accessibility to complex skill training.

Keywords: Extended Reality, Digital Twin, FPV Drone, Human-AI Co-training, PID Control, Reinforcement Learning

Contents

Abstract **2**

Introduction **6**

Theoretical Background **8**

Introduction to Quadrotor Systems 8

Flight Dynamics and Principles of Operation 8

Pilot Interface and Control Systems 10

PID Control 12

Reinforcement Learning 13

Fundamentals of Reinforcement Learning 13

Markov Decision Processes (MDPs) 14

Learning Theories in Simulation-Based XR 15

Experiential Learning 15

Active Learning 15

Game-Based Learning 15

Simulation Games 15

Game Design and Learning Frameworks 15

Game Design Framework 15

Learning Mechanics–Game Mechanics 15

Human–Computer Interaction in XR 15

XR Inputs 15

XR UI Frameworks 15

Digital Twin and Spatial Computing Concepts 15

Geographic Information Systems 15

Digital Twin 15

AI-assisted environment generation 15

Methodology **16**

Design-Based Research 16

System Design Overview 16

Archetecture 16

Models 16

Environment Construction 16

Interaction and Control Design 16

Implementation of Drone Physics: A Case Study of Yue's Ultimate Drone 16

Control Input and Joystick Shaping 17

Flight Modes and Orientation Logic 17

Physics Calculation and PD Torque 18

PD Control 18

Attitude Control 19

Path Generation 20

Cascaded Control Loops 21

Implementation Logic 22

Implementation with Unity ML-Agents 23

The Hovering Task 25

Proximal Policy Optimization (PPO) 25

Tutoring UI 27

Comparison of Current XR Development Platforms **27**

Hardware Architectures: PC VR vs. Standalone 27

Passthrough (video see-through) 29

Software Standards 29

OpenXR 29

WebXR 30

Software Engine Comparison 30

Learning System Design 32
Data Collection Methods 32

Results **33**
Prototype Development Process 33
Drone Control Implementation 33
Physics and Simulation System 33
XR Interaction and UI Results 33

Discussion **34**
Research Questions Revisited 34
Interpretation 34
Connection to Theory 34
Contribution 34
Limitations 34
Future Work 34

Summary **35**
Goals 35
Results 35
Generalization 35
Practical Use 35
Future Study 35

Introduction

Robots are increasingly capable of replacing routine manual and cognitive tasks. However, there remains a strong demand for human cognitive and motor skills that machines cannot yet replicate. At the same time, this technological shift is occurring alongside a growing labor shortage, partly because many workers are unwilling to take jobs that are physically demanding and are expected to be replaced by automation in the future.

As a result, industries such as agriculture, forestry, and heavy-duty mobile machinery are already facing significant shortages of skilled workers (Machado et al., 2021). Moreover, career expectations are increasingly influenced by perceptions of automation. Research shows that when individuals believe a job is likely to be replaced by artificial intelligence, their willingness to pursue that career decreases significantly (Wang et al., 2026).

Despite the growing level of automation, current robotic systems still struggle with complex and dynamic environments. Therefore, skilled human operators remain essential. However, acquiring these skills is difficult and time-consuming, as it requires extensive repeated practice to develop muscle memory.

In this context, learning theory provides a useful framework. Near transfer refers to applying skills in similar contexts, while far transfer refers to applying them in different contexts (Barnett & Ceci, 2002). For near transfer, repeated practice plays a critical role in building procedural knowledge and muscle memory.

Given these challenges, Extended Reality (XR) has emerged as a promising training approach. Previous studies have explored XR in domains such as medical procedures and aviation (Dutescu et al., 2025). Meanwhile, Unmanned Aerial Vehicles (UAVs) are widely used in construction, agriculture, logistics, and disaster management (Otto et al., 2018). Among these applications, First Person View (FPV) quadcopter piloting represents a particularly demanding skill that relies heavily on precise motor control and spatial awareness. Accordingly, this study uses FPV quadcopter piloting as a case to investigate XR-based training for muscle memory development.

However, existing drone simulators are primarily designed for entertainment rather than structured learning. Consequently, they often present steep learning curves and provide limited support for systematic skill acquisition. In addition, the lack of standardized controller configurations create further barriers (Ebeid et al., 2018), leading to user frustration even before effective training begins.

Taken together, low career expectations, high preparation effort, and early-stage frustration significantly reduce learner motivation. As a result, many beginners discontinue training before acquiring sufficient muscle memory.

To address these issues, the goal of this study is to design an XR-based training system that lowers the learning barrier for FPV quadcopter piloting. In particular, the proposed system aims to support muscle memory development through AI-assisted co-training and improved interaction design.

This study addresses the following research questions:

(1) How can FPV quadcopter flight physics be simulated?

(2) How can a human-AI co-training framework be designed to assist users?

(3) What interaction methods enable high-fidelity XR prototyping?

This study uses a design-based research approach. It applies agile development practices and iterative prototyping to build an XR FPV quadcopter simulator, including physics simulation, AI co-training (PID control and reinforcement learning), and interaction design.

Theoretical Background

Introduction to Quadrotor Systems

A quadcopter, also frequently referred to as a quadrotor, is a type of multirotor helicopter or rotary-wing Unmanned Aerial Vehicle (UAV) that utilizes four rotors to achieve flight (Luukkonen, 2011). These rotors are typically directed upwards and arranged in a square formation, with each rotor positioned at an equal distance from the center of mass. In this configuration, the four actuators are generally equally spaced by 90° from one another (Idrissi et al., 2022).

The origins of quadrotor technology date back to 1907, when the Breguet brothers constructed the Gyroplane No. 1. While this marked one of the first manned helicopter flights, it was not a free flight; the machine lacked inherent stability and a proper control mechanism, requiring it to be tethered to the ground (Ghazbi et al., 2016).

The transition to modern UAVs occurred in the late 1990s. A pivotal moment was the 1999 release of the Draganflyer I, the first camera-equipped quadrotor to achieve mass production. This shifted quadrotor technology from a laboratory curiosity to a viable commercial product (Karahan, 2025).

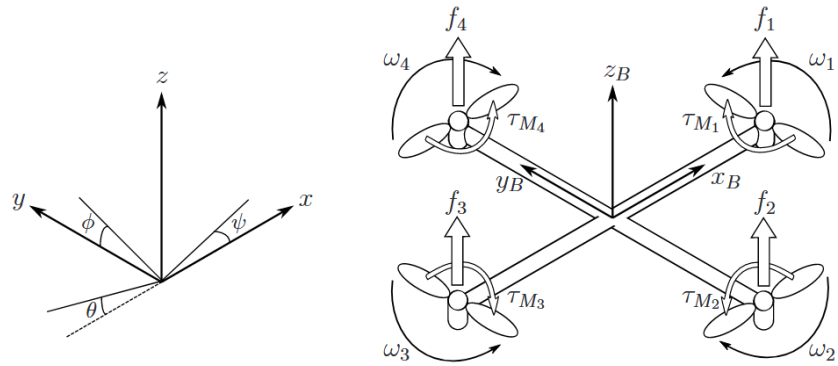
Since 2010, advancements in micro-electronics and control theory have expanded the use of quadrotors across various sectors:

- Public Safety: Search and rescue, firefighting, and border surveillance.
- Agriculture: Precision seeding, chemical spraying, and crop monitoring.
- Commercial: Last-mile delivery and high-end cinematography.

Flight Dynamics and Principles of Operation

A quadcopter is an under-actuated system, meaning it must manage six degrees of freedom (6DOF) as shown in fig. 1, three translational and three rotational—using only four control inputs (the angular velocities of the rotors).

Figure 1
The inertial and body frames of a quadcopter



The fundamental principle of quadcopter stability relies on torque cancellation. To maintain a stable orientation, two motors rotate clockwise (CW) while the adjacent pair rotates counter-clockwise (CCW). This arrangement ensures that the net moment about the center of mass is zero; without this pair-wise opposition, the drone would rotate uncontrollably around its vertical axis. Quadcopters typically use either a plus (+) or cross (x) configuration, with the cross configuration being more common due to its superior stability (Idrissi et al., 2022).

The absolute linear position of the quadcopter is defined in the inertial frame axes ($[x, y, z]^T$). The attitude, i.e.the angular position, is defined in the inertial frame with three Euler angles ($[\phi, \theta, \psi]^T$), which represent pitch angle, roll angle and yaw angle as shown in fig. 1.

Every change in position or attitude is achieved by varying the individual angular velocities (ω_i) of the four rotors:

Thrust (T): Simultaneously increasing all rotor speeds causes the vehicle to ascend. The force generated by rotor i is $f_i = k\omega_i^2$, where k is the lift constant. Total thrust is defined as show in eq. (1)

$$T = k \sum_{i=1}^{4} \omega_i^2 \tag{1}$$

Yaw (τ_ψ): Rotating the drone around the vertical axis is achieved by creating a torque imbalance between the CW and CCW pairs shown in eq. (2), where b is the drag constant.

$$\tau_\psi = b(\omega_1^2 - \omega_2^2 + \omega_3^2 - \omega_4^2) \tag{2}$$

Roll (τ_ϕ) and Pitch (τ_θ): These are achieved by creating a speed differential between opposite rotors. For a distance l from the center of mass, shown in eq. (3)

$$\begin{aligned} \tau_\phi &= lk(-\omega_2^2 + \omega_4^2) \\ \tau_\theta &= lk(-\omega_1^2 + \omega_3^2) \end{aligned} \tag{3}$$

The quadcopter is modeled as a rigid body using Newton-Euler equations. The acceleration in the inertial frame $[x, y, z]^T$ is governed by gravity (g) and the orientation of the thrust vector shown in eq. (4).

$$\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} + \frac{T}{m} \begin{bmatrix} \cos\psi \sin\theta \cos\phi + \sin\psi \sin\phi \\ \sin\psi \sin\theta \cos\phi - \cos\psi \sin\phi \\ \cos\theta \cos\phi \end{bmatrix} \tag{4}$$

In the body-fixed frame, the angular acceleration is influenced by the external torque (τ), inertia matrix (I), and gyroscopic effects as shown in eq. (5)

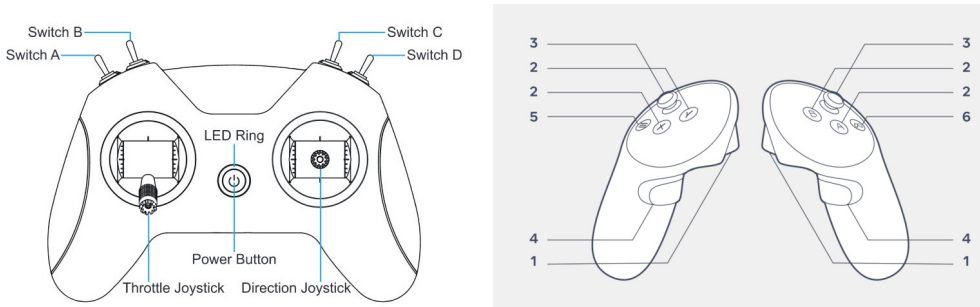
$$\begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} (I_{yy} - I_{zz})qr/I_{xx} \\ (I_{zz} - I_{xx})pr/I_{yy} \\ (I_{xx} - I_{yy})pq/I_{zz} \end{bmatrix} + \begin{bmatrix} \tau_\phi/I_{xx} \\ \tau_\theta/I_{yy} \\ \tau_\psi/I_{zz} \end{bmatrix} \tag{5}$$

Pilot Interface and Control Systems

The First-Person View (FPV) experience is defined by the synchronization between the pilot's inputs and the drone's visual feedback. An FPV drone can be controlled with a traditional remote controller or with a Motion Controller shown in fig. 2. A Motion Controller allows they user to maneuver the drone using hand gestures which is easy and convenient. The FPV Goggles is a also used for the FPV Drone. It's the defining factor for the FPV Drone. It connects to the drone's camera so that the camera user can have full vision of the quadcopter's camera (Usama

et al., 2021).

Figure 2
Transmitter (Left) and Gamepad (Right)



Adapted from (BetaFPV, 2022)

The industry standardizes these controls into two primary layouts: Mode 1 (Japanese Hand): Predominantly used in Japan and parts of Europe. The throttle is located on the right gimbal; Mode 2 (American Hand): The global standard for FPV. The throttle is located on the left gimbal, which most pilots find more intuitive for separating altitude from directional pitch. Difference between Mode 1 and Mode 2 is shown in table 1

From a signal processing perspective, the flight controller differentiates between the unipolar throttle and the bipolar command sticks: Throttle (T): Mapped to a range of $[0, 1]$. Roll, Pitch, and Yaw (ϕ, θ, ψ): Mapped to a range of $[-1, 1]$. In fig. 3 shows the interaction of the transmitter in Mode 2 and how the quadcopter move.

FPV drones generally operate across two flight logic modes. Self-Leveling (Angle/Normal Mode): The onboard flight controller uses accelerometers to automatically return the drone to a level horizon when the sticks are centered. This is ideal for beginners but limits maneuverability; Acrobat (Manual Mode): The controller only uses gyroscopes to maintain the current angular rate. If the pilot moves the stick and returns it to center, the drone maintains its current tilted orientation. This allows for flips and rolls but requires constant manual correction.

The FPV goggles provide the pilot with a low-latency first person view video feed pass from the camera set on the quadcopter's front. Beyond the video feed, an On-Screen Display

Table 1
Comparison of Transmitter Control Modes

Mode	Left Stick (V)	Left Stick (H)	Right Stick (V)	Right Stick (H)
Mode 1	Pitch	Yaw	Throttle	Roll
Mode 2	Throttle	Yaw	Pitch	Roll

(OSD) provides critical telemetry data, including battery voltage, signal strength, and flight time, as shown in fig. 4.

PID Control

In 1922, Nicolas Minorsky presented a very clear analysis of the control actions necessary to provide effective control of a system whose exact dynamics were un- known. He analyzed the actions taken by a good helmsman steering a ship and translated these actions into the appropriate mathematical formulations (Bennett, 1993). It is now know as the Proportional-Integral-Derivative (PID) controller. PID is a type of controller that is most commonly used and applied in mechanical systems (Idrissi et al., 2022).

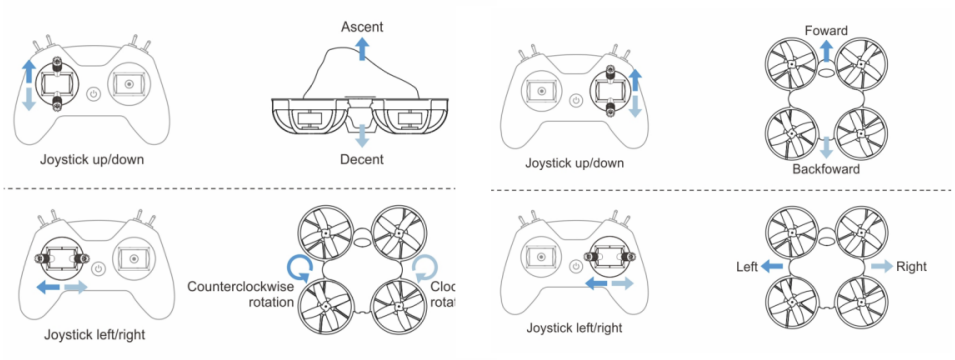
The controller consists of three main components:

- Proportional (K_p): Acts like a virtual spring that pulls the system toward the desired position; the control action increases as the error gets larger.
- Integral (K_i): Accumulates the error over time to eliminate steady-state error, ensuring the drone reaches the exact target even in the presence of disturbances like gravity.
- erivative (K_d): Acts as a virtual damper to reduce oscillations and prevent the system from overshooting its target.

The idealized control law is expressed as:

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt}$$

Figure 3
Mode 2 control system



Adapted from (BetaFPV, 2022)

Reinforcement Learning

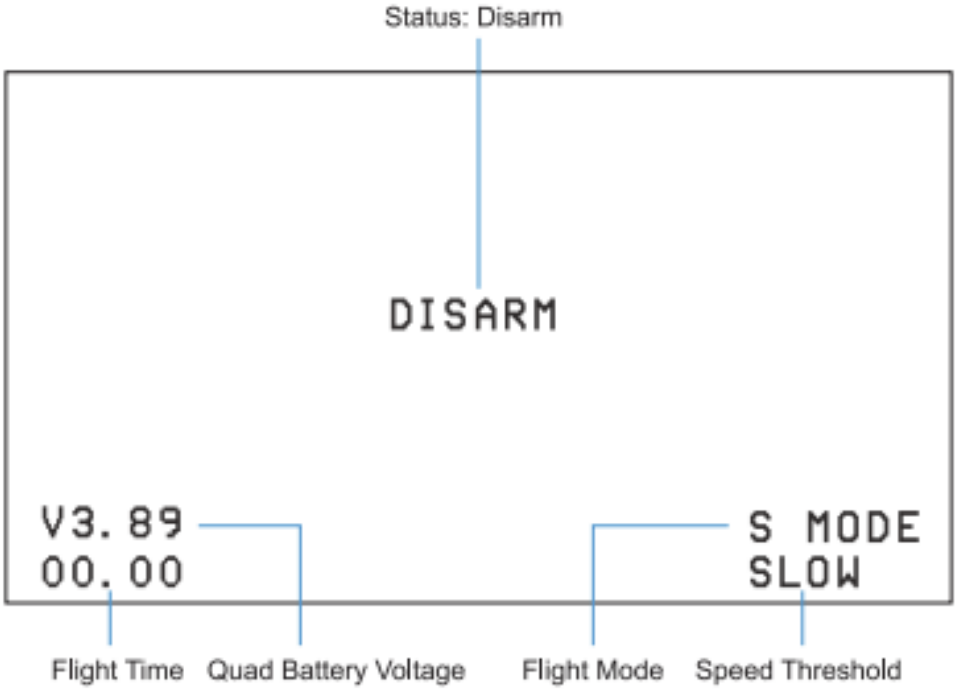
While Proportional-Integral-Derivative (PID) controllers are the industry standard for stable piloting—offering a simple, "white-box" solution—they often struggle with complex, non-linear dynamics where the Integral term alone is insufficient. Recently, Reinforcement Learning (RL) has gained traction as a robust "black-box" alternative. To reduce the computational burden on onboard systems, reinforcement learning has increasingly been adopted as a key approach for autonomous obstacle avoidance (Yu et al., 2025).

While many RL solutions bridge the gap from computer vision to control, this chapter explores a "target-to-target" approach. Using Unity's ML-Agents toolkit, I aim to develop a training pipeline that investigates the efficacy of RL for precise drone hovering.

Fundamentals of Reinforcement Learning

The concept of implementing RL in computing is among the earliest visions of AI. In a 1948 report, Alan Turing described a design for a pleasure-pain system (Barto, 2019). Modern RL defines a framework where an agent learns behavior through trial-and-error interactions with a dynamic environment (Kaelbling et al., 1996). Unlike supervised learning, the agent is not provided with "correct" answers. Instead, it receives a scalar reinforcement signal (reward or punishment) and must discover actions that maximize its long-term cumulative reward.

Figure 4
The FPV goggles interface



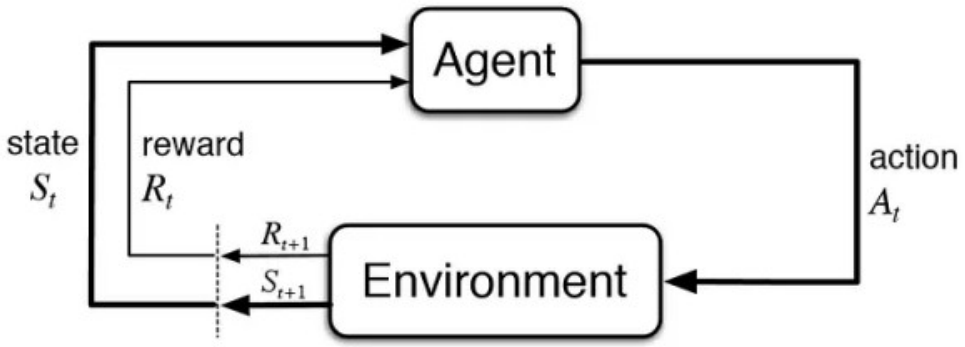
Adapted from (BetaFPV, 2022)

Markov Decision Processes (MDPs)

MDPs provide the mathematical foundation for RL fig. 5. In an MDP, an agent's actions influence both the immediate reward and the subsequent state of the environment. An MDP is formally defined by:

- A set of states (S): Possible environment configurations.
- A set of actions (A): Choices available to the agent.
- A reward function (R): $R : S \times A \to \mathbb{R}$, specifying the instantaneous reward.
- A state transition function (T): $T : S \times A \to \Delta(S)$, defining the probability of transitioning to state s' given action a in state s.

Figure 5
Markov Decision Process diagram



Learning Theories in Simulation-Based XR

Experiential Learning

Active Learning

Game-Based Learning

Simulation Games

Game Design and Learning Frameworks

Game Design Framework

Learning Mechanics–Game Mechanics

Human–Computer Interaction in XR

XR Inputs

XR UI Frameworks

Digital Twin and Spatial Computing Concepts

Geographic Information Systems

Digital Twin

AI-assisted environment generation

Methodology

Design-Based Research

System Design Overview

Archetecture

Models

Environment Construction

Interaction and Control Design

Implementation of Drone Physics: A Case Study of Yue's Ultimate Drone

This section examines the flight dynamics and control implementation found in the "Yue's Ultimate Drone" fig. 6 physics model. The system architecture separates pilot input normalization from the underlying Proportional-Derivative (PD) control loops.

Figure 6
Yue's Ultimate Drone

Control Input and Joystick Shaping

The flight controller distinguishes between unipolar throttle inputs and bipolar command sticks (Roll, Pitch, and Yaw):

- Throttle (T): Mapped to a normalized range of $[0,1]$.
- Attitude Commands (θ, ϕ, ψ): Mapped to a range of $[-1,1]$.

To refine the pilot's control over the aircraft, the system utilizes a non-linear control curve for joystick calculation. As Lovesay (2003) observes, a good quality force tracking controller has a linear output, with joystick force directly proportional to sight velocity, but many systems shape the joystick voltage output, to achieve fine control and greater tracking accuracy as the joystick output approaches zero, whilst retaining the same maximum output capability.

In this implementation, "shaping" is achieved by combining proportional (p) and exponential (e) components to calculate the target angular rate (τ) from the raw angular rate (u):

$$\tau = p \cdot u + e \cdot \mathrm{sgn}(u) \cdot u^2$$

The specific rates for the rotational axes are calculated as follows:

- Yaw Rate (τ_ψ): $\tau_\psi = p \cdot u_\psi + e \cdot \mathrm{sgn}(u_\psi) \cdot u_\psi^2$
- Roll Rate (τ_ϕ): $\tau_\phi = p \cdot u_\phi + e \cdot \mathrm{sgn}(u_\phi) \cdot u_\phi^2$
- Pitch Rate (τ_θ): $\tau_\theta = p \cdot u_\theta + e \cdot \mathrm{sgn}(u_\theta) \cdot u_\theta^2$

The raw gamepad throttle $u_T \in [-1,1]$ is normalized to a linear thrust scalar:

$$T = \frac{u_T + 1}{2}$$

Flight Modes and Orientation Logic

Since the Unity engine utilizes the FixedUpdate method for physics calculations, the drone's target orientation is updated discretely at every physics timestep (Δt).

In Acro mode, the joystick inputs command the angular rate of change. The drone's target orientation q_{target} is an accumulation of these rates over time. The orientation is updated by integrating the processed angular rates $(\tau_\phi, \tau_\theta, \tau_\psi)$:

$$q_\Delta = \text{Quaternion.Euler}(\tau_\theta \Delta t, \tau_\phi \Delta t, \tau_\psi \Delta t)$$

$$q_{\text{target},k} = q_{\text{target},k-1} \cdot q_\Delta$$

In Self-Leveling mode, intended to prevent crashing and ease navigation, the pitch and roll sticks command a specific angle rather than a rate. Given a predefined maximum bank angle for pitch (θ_{max}) and roll (ϕ_{max}), the target values are calculated as:

$$\theta_{\text{target}} = u_\theta \cdot \theta_{\text{max}}, \quad \phi_{\text{target}} = u_\phi \cdot \phi_{\text{max}}$$

The yaw axis typically remains in rate-integration mode to allow for continuous 360-degree rotation:

$$q_{\text{target}} = \text{Quaternion.Euler}(\tau_\theta \cdot \theta_{\text{max}}, \tau_\phi \cdot \phi_{\text{max}}, \tau_\psi \Delta t)$$

Physics Calculation and PD Torque

The simulation applies two primary vectors to the drone's 'Rigidbody': 'appliedForce' (Thrust) and 'appliedTorque' (Attitude Control).

The total thrust force $\mathbf{F}_{\text{thrust}}$ is applied along the vehicle's local "up" vector ($\hat{\mathbf{u}}_{\text{up}}$). The magnitude is governed by the throttle T and the maximum thrust constant F_{max}:

$$\mathbf{F}_{\text{thrust}} = (T \cdot F_{\text{max}}) \cdot \hat{\mathbf{u}}_{\text{up}}$$

PD Control

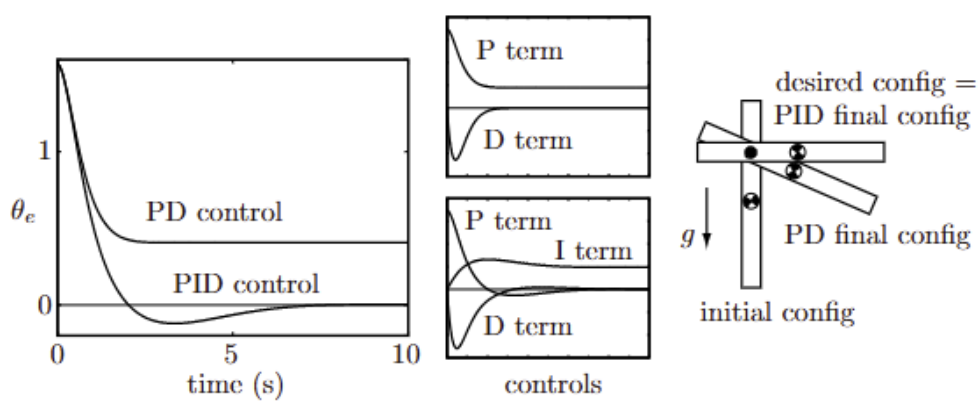
Proportional integral derivative (PID) fig. 7 is a type of controller that is most commonly used and applied in mechanical systems (Idrissi et al., 2022).

The controller consists of three main components:

- Proportional (K_p): Acts like a virtual spring that pulls the system toward the desired position; the control action increases as the error gets larger.
- Integral (K_i): Accumulates the error over time to eliminate steady-state error, ensuring the drone reaches the exact target even in the presence of disturbances like gravity.
- Derivative (K_d): Acts as a virtual damper to reduce oscillations and prevent the system from overshooting its target.

$$u(t) = K_p e(t) + K_i \int e(t)dt + K_d \frac{de(t)}{dt}$$

Figure 7
Comparison of PID and PD



For basic flight stabilization where the full dynamic behavior of the plant is not realized, a simple PD action can provide satisfactory performance

Attitude Control

The error between the target orientation q_{target} and the current body orientation q_{body} is defined as:

$$\Delta q = q_{\text{target}} \cdot q_{\text{body}}^{-1}$$

Using Unity's 'ToAngleAxis' method, this relative rotation is converted into an angle ϕ and an axis $\mathbf{n}$. The error vector $\mathbf{e}$ is then:

$$\mathbf{e} = \phi \cdot \mathbf{n}$$

The controller applies a PD loop (Proportional-Derivative) to generate corrective torque.

$$\tau_{\text{world}} = K_p \cdot \mathbf{e} + K_d \cdot \frac{\mathbf{e} - \mathbf{e}_{k-1}}{\Delta t}$$

Finally, the world-space torque is transformed into the body frame (R_{wb}) for the physics simulation:

$$\tau_{\text{body}} = R_{\text{wb}} \cdot \tau_{\text{world}}$$

Path Generation

The objective of this chapter is to implement a closed-loop mission controller. This system translates high-level world-frame goals (waypoint coordinates, target altitude, and yaw) into RC-style stick commands (normalized -1 to 1) compatible with the existing Yue drone physics engine.

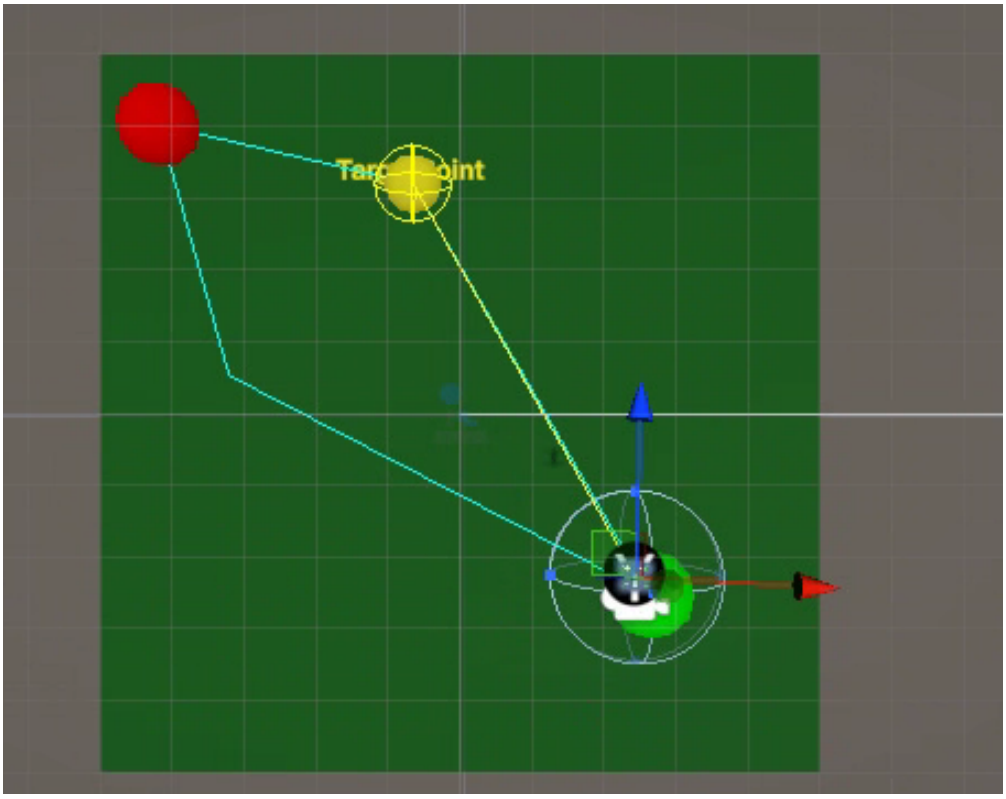
To move between points, the agent generates a path using two Cubic Bézier curves. Given four control points P_0, P_1, P_2, P_3, home to waypoint and back fig. 8, the position $B(t)$ for $t \in [0,1]$ is defined as:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$$

The path is constructed relative to the "Home" (P_{home}) and "Target" (P_{target}) vectors:

- Forward Vector: $\vec{f} = \text{normalize}(P_{target} - P_{home})$
- Orthogonal (Right) Vector: $\vec{r} = \vec{up} \times \vec{f}$ (This handles the lateral expansion of the teardrop).

Figure 8
Path Design



- Bulge Parameterization: A ‘widthRatio‘ variable allows the path to scale from a narrow line to a wide teardrop shape.

The mission follows a two-phase approach to ensure a continuous loop:

- Outbound: Path from Home to Waypoint using a right-hand offset for control points.
- Inbound: Path from Waypoint back to Home using a left-hand offset.

Cascaded Control Loops

The system employs a cascaded control scheme where the Outer Loop (Position) provides setpoints for the Inner Loop (Attitude/Velocity).

- The Inner Loop features controllers that yield the required rolling and pitching moments τ_x and τ_y to regulate horizontal movement. The Inner Loop block requires the current

quadcopter attitude as well as reference signals for roll and pitch to obtain its output (Paredes et al., 2021).

- The Outer Loop, which features controllers that yield the required thrust differential and yawing moments as well as the desired reference signals.

Implementation Logic

During takeoff, the controller focuses exclusively on vertical error (e_y). The vertical force $forcey_Y$ is mapped to a normalized throttle value (T):

$$T = \frac{u_T + 1}{2}$$

Following a path requires a look-ahead method. Given current position **p** and look-ahead target **t** on the path, the position error is defined as:

$$e_x = t_x - p_x, \quad e_y = t_y - p_y, \quad e_z = t_z - p_z$$

The Outer Loop converts this error into a Desired Velocity in the world axis:

$$v_{x,\text{des}} = \text{PID}_X(e_x), \quad v_{z,\text{des}} = \text{PID}_Z(e_z)$$

To translate these into drone-relative commands (Pitch/Roll), the world velocity is projected onto the drone's Heading Axis:

- Forward Basis: $\hat{\mathbf{f}} = R_y(\psi)\,\hat{\mathbf{z}}_{\text{forward}}$
- Right Basis: $\hat{\mathbf{r}} = R_y(\psi)\,\hat{\mathbf{x}}_{\text{right}}$
- Projections:

$$v_{\text{des}}^{\parallel} = \mathbf{v}_{H,\text{des}} \cdot \hat{\mathbf{f}}, \quad v_{\text{des}}^{\perp} = \mathbf{v}_{H,\text{des}} \cdot \hat{\mathbf{r}}$$

The Inner Loop then calculates the error between desired and actual velocity to produce stick inputs:

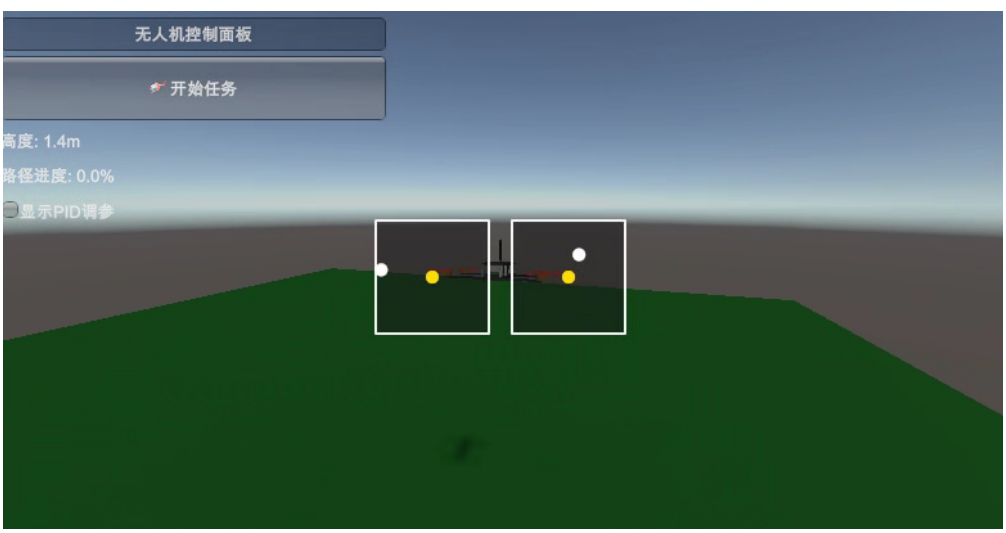
- Pitch (Right Vertical): Based on longitudinal velocity error ($e_{v,\parallel}$).
- Roll (Right Horizontal): Based on lateral velocity error ($e_{v,\perp}$).

The drone should yaw toward its movement direction once a "velocity threshold" (0.5 m/s) is met.

- Target Yaw: $\psi_{\text{target}} = \text{atan2}(v_x, v_z)$
- Yaw Error: $e_\psi = \text{DeltaAngle}(\psi, \psi_{\text{target}})$
- Output: $u_{\text{yaw}} = \text{PID}_{\text{yaw}}(e_\psi)$

By calculating these normalized joystick rates, the system can visualize the "AI's intent" on the HUD. This serves as a co-training hinter, showing human pilots the optimal stick positions for maintaining the designated path fig. 9.

Figure 9
hinter UI



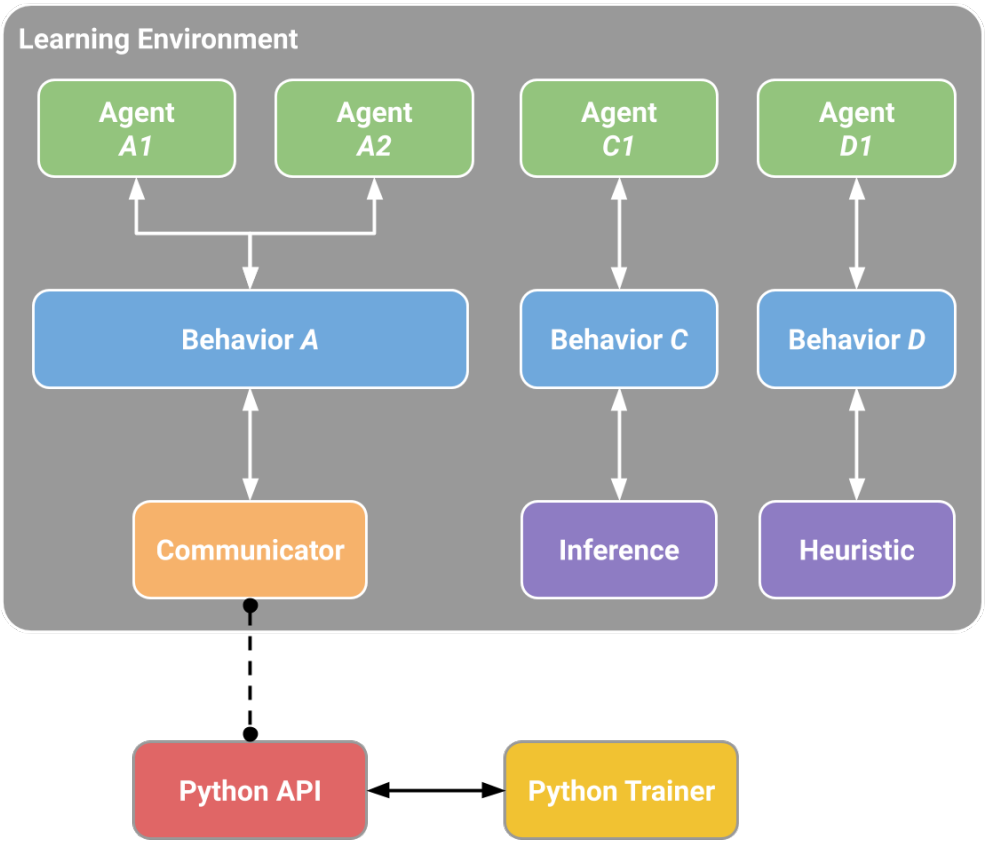
Implementation with Unity ML-Agents

ML-Agents fig. 10 enable games and simulations to serve as environments for training intelligent agents in Unity. Training can be done with reinforcement learning, imitation learning, neuroevolution, or any other methods. Trained agents can be used for many use cases, including

controlling NPC behavior (in a variety of settings such as multi-agent and adversarial), automated testing of game builds and evaluating different game design decisions pre-release.

Every Learning Environment will always have one Agent for every character in the scene. While each Agent must be linked to a Behavior, it is possible for Agents that have similar observations and actions to have the same Behavior.

Figure 10
Simplified ML-Agents Scene Block Diagram



For this research, each Drone Agent includes:

- Agent Script: Handles ‘OnActionReceived‘ (processing "joystick" inputs), ‘CollectObservations‘ (monitoring states), and ‘Heuristic‘ (manual control for debugging).
- Behavior Parameters: Defines the vector space for actions and observations.
- Decision Requester: Controls the frequency of actions. By limiting the request rate, we can

simulate more "human-like" reaction times, making the agent a better tutor for human pilots.

The Hovering Task

We implemented a 1D control mission: regulating the drone's vertical position (y) within a target band $[H-R, H+R]$, where H is the 'targetHeight' and R is the 'acceptableRadius'.

The following two values are being observed.

- Normalized Height Error: $\text{clip}(\frac{y-H}{R}, -k, k)/k$ (where k is 'errorClampK'). This maps the error to a $[-1, 1]$ range, preventing extreme spikes from destabilizing the gradient.
- Vertical Velocity: v_y/v_{ref}, clipped to $[-1, 1]$.

The agent uses one continuous action in the range $[0, 1]$, representing the throttle input. Rewards are calculated at each decision step (Δt):

- Event-based: A 'firstArrivalReward' is granted upon entering the target band for the first time per episode.
- Dense/Steady-state: A 'livingRewardPerSecond' is granted while maintaining position within the band.
- Shaping: A penalty proportional to the normalized excess $(|e|-R)/R$ is applied when outside the band to discourage drifting.

Proximal Policy Optimization (PPO)

We utilized Proximal Policy Optimization (PPO), an on-policy actor-critic algorithm that updates policies in small, clipped steps to ensure stability (OpenAI, 2017). PPO is particularly effective for drones learning to navigate complex environments (Yu et al., 2025).

The optimization objective is defined as:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t[\min(r_t(\theta)\hat{A}_t, \text{textclip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t)]$$

- θ is the policy parameter
- $\hat{E}_t$ denotes the empirical expectation over timesteps
- r_t is the ratio of the probability under the new and old policies, respectively
- $\hat{A}_t$ is the estimated advantage at time t
- ε is a hyperparameter, usually 0.1 or 0.2

The agent was trained for 5×10^5 steps using the following hyperparameters:

- Clipping parameter $\varepsilon = 0.2$
- GAE parameters $(\lambda, \gamma) = 0.99$
- Neural network: a 2-layer MLP with 128 hidden units
- Learning rate: 3×10^{-4} with linear decay

To monitor progress, we implemented a color-coded feedback system, fig. 11:

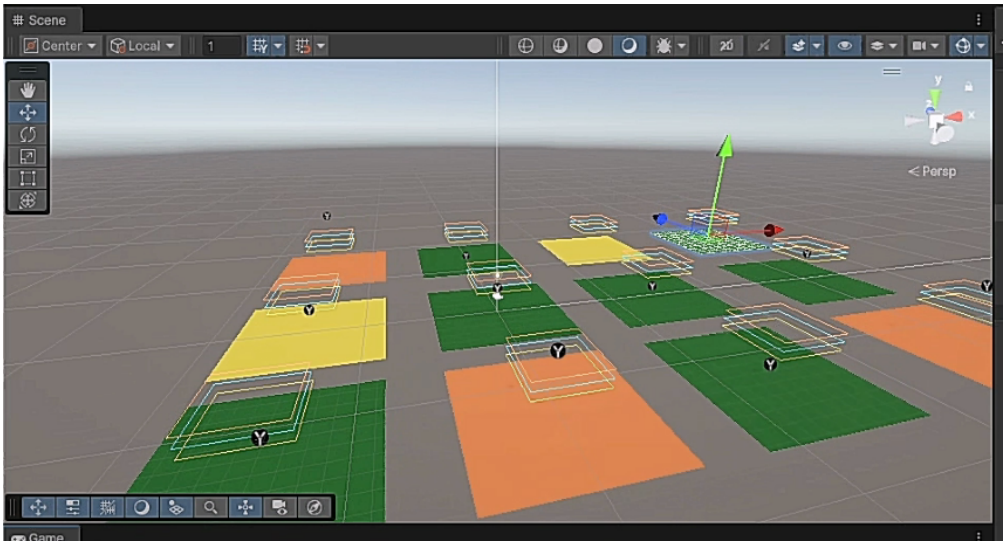
- Yellow: Below target.
- Orange: Above target.
- Blue: Stable hovering.

Cumulative Reward: Showed steady growth after 100k steps, plateauing at 400k, fig. 12.

Episode Length: Positively correlated with rewards; as the agent improved, it survived longer without crashing (falling too low).

Environment/Cumulative Reward Histograms: also shows when training time raises, reward start to raise.

Figure 11
trainig process



Tutoring UI

The RL agent successfully learned to hover with high precision. By adjusting the ‘DecisionRequester‘, we created a smoother control hint for the UI, matching the pace of human perception fig. 13.

Comparison of Current XR Development Platforms

XR development is heavily dictated by the choice of game engine. Before initiating a project, it is critical to determine the target hardware and environmental requirements, as these factors define the development constraints.

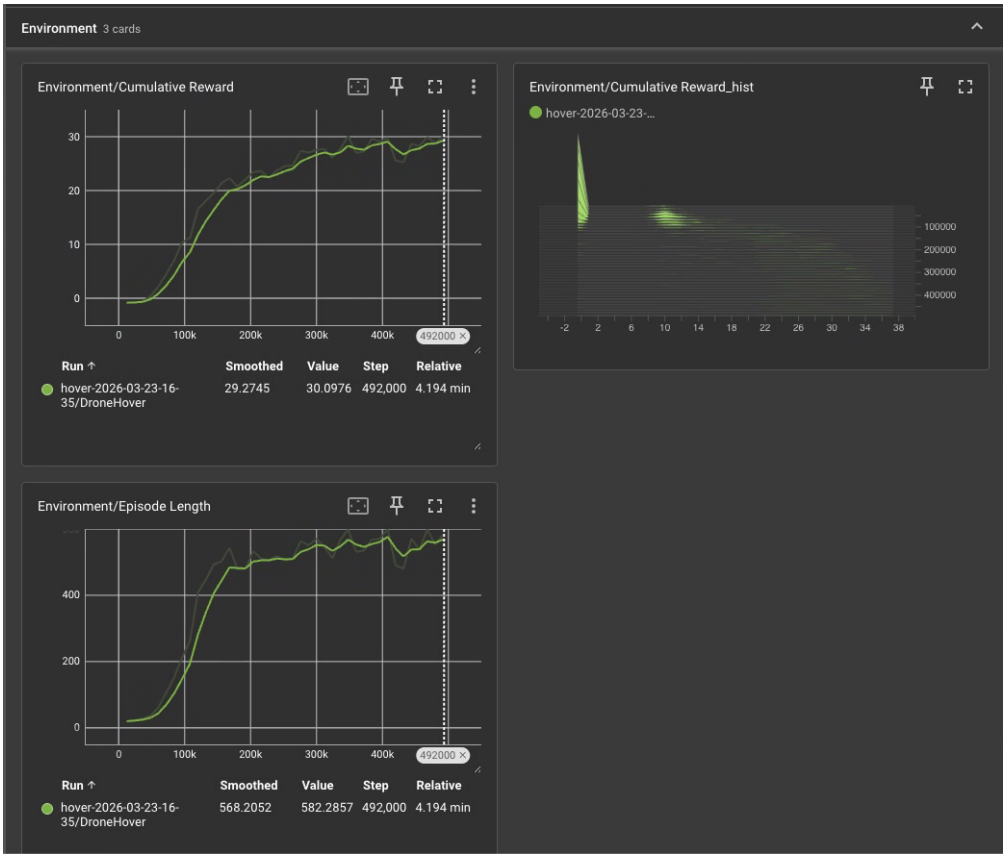
Due to hardware availability, this paper focuses exclusively on development for the Meta Quest 3.

Hardware Architectures: PC VR vs. Standalone

For Head-Mounted Devices (HMDs) like the Meta Quest, two primary architectural configurations exist:

PC VR (Tethered): The headset acts as a peripheral. Assets are rendered on a high-end PC and streamed to the device via a wired (Link) or wireless (Air Link) connection. This allows developers to utilize the full power of engines like Unreal Engine, enabling high-fidelity features

Figure 12
Result



such as Lumen and Nanite. PC VR also supports specialized peripherals like haptic suits or omnidirectional treadmills, as seen in industry-standard titles like Half-Life: Alyx.

Standalone VR (All-in-One): Standalone headsets are self-contained systems with internal mobile chipsets. These provide high mobility but require significant optimization. High-end features must often be disabled or simplified to maintain performance. However, lightweight engines like Unity or Godot excel in this space, offering a balance between visual quality and mobile performance.

Rendevski et al. (2022) note that Standalone VR (SVR) applications offer significant advantages over PC VR. With proper optimization techniques, almost any PC VR scenario can be developed and deployed as SVR while still providing an acceptable user experience.

Differently, a recent survey from the Game Developers Conference (GDC) shows that

among developers interested in creating VR/AR/MR games, the SteamVR platform (PC VR) slightly surpassed the Meta Quest / Horizon Store (68% vs. 64%) as their platform of interest. These were followed by PlayStation VR/VR2 (34%) and Apple visionOS (15%). However, when filtering for developers who have actually worked on VR/AR/MR games, a strong preference remains for the Meta Quest (72%) over SteamVR (59%).

Passthrough (video see-through)

Mixed reality (MR) headsets create immersive experiences designed to spatially integrate virtual content into the physical world. The newest headsets rely on passthrough video. While using passthrough, a person does not see light from the real world but instead relies on stereoscopic, color, high resolution, low latency, real-time video of the world, which is displayed on small screens inside a headset.(Bailenson et al., 2024)

Waldow et al. (2025) investigates Pass-Through Embodiment (PTE), The study found that PTE significantly enhances the user's sense of presence and embodiment compared to using a customizable digital avatar. The ability to see one's own body strengthens the feeling of being physically "present" in the VR environment without negatively affecting performance or causing sickness.

Leveraging passthrough on modern XR devices can help developers build highly engaging mixed-reality environments, which is particularly beneficial for applications like prototyping drone simulations.

Software Standards

To ensure cross-platform compatibility, developers rely on unified industry standards. Currently, there are two major public standards:

- OpenXR - WebXR

OpenXR

OpenXR is the industry-standard API for XR applications. It is the interface between an application and an in-process or out-of-process "XR runtime system". The runtime system handles functionality such as frame composition, peripheral management, and raw tracking

information (Khronos OpenXR Working Group, 2024). Major game engines and the Meta Spatial SDK are built upon OpenXR to ensure hardware-agnostic development.

A primary bottleneck in XR development is the "deployment loop." Compiling a standalone APK and pushing it to a device can take 10–15 minutes per iteration.

- Link Mode: Developers often use the Oculus Link App to preview scenes in PC VR, though this requires constantly putting on and taking off the headset.

- Simulators: The Meta XR Simulator is increasingly becoming the preferred method for rapid iteration, allowing developers to simulate head and controller movements directly on their PC without needing physical hardware.

WebXR

The WebXR Device API provides the interfaces necessary to enable developers to build compelling, comfortable, and safe immersive applications on the web across a wide variety of hardware form factors (World Wide Web Consortium, 2022).

WebXR serves as a higher-level abstraction that runs in the browser, combining older specifications from WebVR and WebAR. The WebXR standard shares core features with and can be viewed as a subset of OpenXR.

While traditional game engines offer WebXR export support, there are fewer dedicated development stacks built exclusively for it. This section focuses specifically on engines that directly call the browser API.

For security reasons, WebXR APIs are restricted to HTTPS. Consequently, developers must use port-forwarding technologies during local development. Thanks to the efficiency of the web stack, a WebXR application typically only requires a few seconds to bundle. It is worth noting, however, that web rendering technology is currently transitioning from WebGL2 to WebGPU, which may introduce API changes in the WebXR ecosystem.

Software Engine Comparison

- Unreal Engine: UE is known for photorealistic rendering, UE provides unparalleled visual fidelity. Its Chaos Physics engine is highly regarded for complex simulations, making it the

Feature	Unreal Engine (5.7)	Unity (6.0)
Primary Language	C++ / Blueprints	C#
Physics Engine	Chaos Physics	NVIDIA PhysX / DO
Rendering Tech	Lumen (Dynamic GI) & Nanite (Virtual Geometry)	URP / HDRP
Meta Support	Fork build	First-class Support
Machine Learning Support	Learning Agent	ML Agents
Editor Size (XR plugin included)	∼50GB+	∼20GB

Table 2
Comparison of game engines

foundation for projects like Microsoft's AirSim. However, UE has a high barrier to entry: the editor is massive, and utilizing Meta's private APIs often requires compiling a custom engine branch from source.

- Unity: Unity remains the most popular choice for Meta Quest development due to its mature SDK support and C# scripting. While it uses PhysX by default, its DOTS (Data-Oriented Technology Stack) provides high-performance physics for complex drone swarms. It offers the most streamlined "out-of-the-box" experience for Standalone VR.

- Godot: Godot has emerged as a powerful, lightweight alternative. It is highly compatible with modern AI coding assistants due to its Python-like GDScript. The integration of Jolt Physics as the default 3D engine has resolved previous performance hurdles for VR. However, its XR ecosystem is largely community-maintained. While excellent for prototyping, the lack of official Meta API support and the potential instability of third-party GDExtensions require careful dependency management for production-grade projects.

- Wonderland Engine: Wonderland Engine is a specialized WebXR engine built in C++ and compiled to WebAssembly. By leveraging WebGL2, it achieves the performance necessary to render thousands of objects at native frame rates directly within a browser. . The development workflow relies on JavaScript and TypeScript, providing a highly familiar and accessible environment for traditional web developers. A unique advantage is the ability to edit scenes directly on the headset, enabling instant feedback without removing the device.

Learning System Design

Data Collection Methods

Results

Prototype Development Process

Drone Control Implementation

Physics and Simulation System

XR Interaction and UI Results

Discussion

Research Questions Revisited

XR supports training; AI provides guidance; system design partially validated.

Interpretation

XR enables immersion; AI provides feedback; combined supports learning.

Connection to Theory

Supports experiential and feedback-based learning.

Contribution

- XR-based training system
- AI-assisted co-training approach
- Integration of gamification and simulation

Limitations

No user study, limited RL, simplified tasks.

Future Work

User evaluation, more complex tasks, improved AI.

Summary

Goals

Develop XR training system with AI guidance.

Results

Functional system with immersive interaction.

Generalization

XR supports skill training.

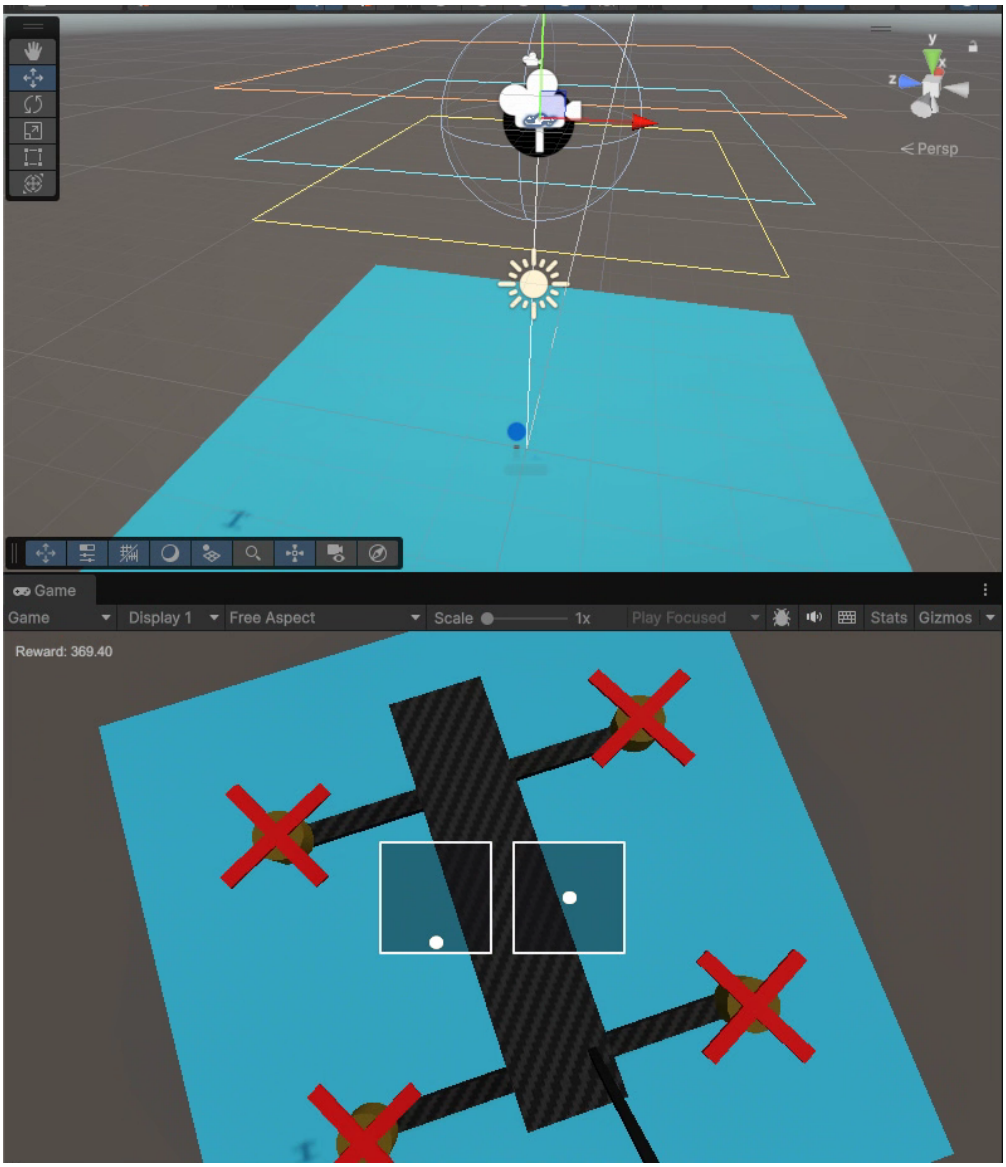
Practical Use

Prototype training framework.

Future Study

Learning validation and system expansion.

Figure 13
Result



References

Barnett, S. M., & Ceci, S. J. (2002). When and where do we apply what we learn?: A taxonomy for far transfer. *Psychological Bulletin*, *128*(4), 612–637. https://doi.org/10.1037/0033-2909.128.4.612

Bennett, S. (1993). Development of the pid controller. *IEEE Control Systems*, *13*(6), 58–62. https://doi.org/10.1109/37.248006

BetaFPV. (2022). *Literadio 3 radio transmitter user manual*. https://support.betafpv.com/hc/en-us/article_attachments/55907267618073

Dutescu, R., Apostol, M., Mocanu, I., & Barbu, M. (2025). Virtual reality in training: A multidimensional analysis of technologies, challenges and paradigms. *Proceedings of the International Conference on Virtual Learning - VIRTUAL LEARNING - VIRTUAL REALITY (20th edition)*, 129–142. https://doi.org/10.58503/icvl-v20y202511

Ebeid, E., Skriver, M., Terkildsen, K. H., Jensen, K., & Schultz, U. P. (2018). A survey of open-source UAV flight controllers and flight simulators. *Microprocessors and Microsystems*, *61*, 11–20. https://doi.org/10.1016/j.micpro.2018.05.002

Ghazbi, S. N., Aghli, Y., Alimohammadi, M., & Akbari, A. A. (2016). Quadrotors unmanned aerial vehicles: A review. *International Journal on Smart Sensing and Intelligent Systems*, *9*(1), 309–333. https://doi.org/10.21307/ijssis-2017-872

Idrissi, M., Salami, M., & Annaz, F. (2022). A review of quadrotor unmanned aerial vehicles: Applications, architectural design and control algorithms. *Journal of Intelligent & Robotic Systems*, *104*(2), 22. https://doi.org/10.1007/s10846-021-01527-7

Karahan, M. (2025). Development stages of quadrotors from past to present: A review. *Drones*, *9*, 840. https://doi.org/10.3390/drones9120840

Luukkonen, T. (2011). Modelling and control of quadcopter.

Machado, T., Fassbender, D., Taheri, A., Eriksson, D., Gupta, H., Molaei, A., Forte, P., Rai, P. K., Ghabcheloo, R., Makinen, S., Lilienthal, A. J., Andreasson, H., & Geimer, M. (2021). Autonomous heavy-duty mobile machinery: A multidisciplinary collaborative challenge.

2021 IEEE International Conference on Technology and Entrepreneurship (ICTE), 1–8. https://doi.org/10.1109/ICTE51655.2021.9584498

Otto, A., Agatz, N., Campbell, J., Golden, B., & Pesch, E. (2018). Optimization approaches for civil applications of unmanned aerial vehicles (UAVs) or aerial drones: A survey. *Networks*, *72*(4), 411–458. https://doi.org/10.1002/net.21818

Paredes, J., Sharma, P., Ha, B., et al. (2021). Development, implementation, and experimental outdoor evaluation of quadcopter controllers for computationally limited embedded systems. *arXiv preprint arXiv:2105.14231*. https://doi.org/10.48550/arXiv.2105.14231

Usama, A., Dora, M., Elhadidy, M., Khater, H., & Alkelany, O. (2021). First person view drone - fpv. *The International Undergraduate Research Conference*, *5*(5), 437–440. https://journals.ekb.eg/article_246400.html

Wang, H., Lai, X., Huang, S., Dai, X., Zhao, X., & Wang, Y. (2026). The influence of prior perception, attitude, and immediate knowledge of AI on adolescents' preferences for high- and low-replaceable jobs. *Behavioral Sciences*, *16*(1), 72. https://doi.org/10.3390/bs16010072